Programación de Sistemas Telemáticos

TLMG1037 · Carrera de Mecatrónica · Propuesta de rediseño (documento interno de trabajo)

1. Descripción

Esta asignatura de formación profesional teórico-práctica de nivel intermedio brinda al estudiante de Mecatrónica las bases de redes, software y conectividad necesarias para desarrollar sistemas telemáticos end-to-end. Se abordan los fundamentos de redes y protocolos de comunicación, el desarrollo de aplicaciones web y APIs para el monitoreo y control remoto, las tecnologías de conectividad inalámbrica de corto y largo alcance, los protocolos de telemetría, los distintos modelos de bases de datos y el despliegue de sistemas en la nube. La asignatura se desarrolla en torno a un proyecto orientado a un problema real, incorpora el uso responsable de herramientas de desarrollo asistido por inteligencia artificial y hace énfasis transversal en la seguridad de la información.

2. Objetivos

1. Aplicar los fundamentos de redes TCP/IP y del protocolo HTTP para la interconexión de dispositivos con servicios remotos.
2. Desarrollar aplicaciones web y APIs para el monitoreo y control remoto de sistemas mecatrónicos, utilizando herramientas de desarrollo asistido por IA y plataformas de despliegue.
3. Seleccionar tecnologías de conectividad inalámbrica y protocolos de telemetría según los requerimientos de alcance, consumo, ancho de banda y costo de cada aplicación.
4. Implementar modelos de almacenamiento de datos adecuados para telemetría y aplicaciones, y desplegar el sistema completo aplicando prácticas de seguridad.

3. Resultados

- Habilidad para adquirir y aplicar nuevos conocimientos de Ingeniería en Mecatrónica según sean necesarios, usando estrategias apropiadas de aprendizaje.
- Diseñar soluciones reales de Ingeniería en Mecatrónica que proponen valor único como respuesta a necesidades específicas consideradas desde el punto de vista de los involucrados.

4. Contenido y horas

Contenido	Horas
Actividades de evaluación	4
Unidad 1. Redes para sistemas conectados	6
Unidad 2. Desarrollo web y APIs REST	7
Unidad 3. Conectividad inalámbrica y telemetría	8
Unidad 4. Datos, integración y despliegue	7
Total	**32**

5. Unidades y subunidades

Unidad 1. Redes para sistemas conectados

Semanas 2–3

Da los fundamentos de redes para entender cómo un dispositivo se comunica con un servidor. Prepara la base conceptual para construir la aplicación en la Unidad 2.

1.1 Sistemas telemáticos y arquitectura end-to-end

Presenta qué es un sistema telemático y sus capas: dispositivo, red, servidor, datos y aplicación. Los equipos definen el problema real que resolverá su proyecto.

1.2 Modelo TCP/IP y direccionamiento: IP, máscara, gateway, DHCP

Explica cómo se organiza una red y cómo un dispositivo obtiene su dirección. El estudiante debe poder diagnosticar por qué un equipo no tiene conexión.

1.3 Servicios de red: DNS, puertos y NAT

Cubre cómo se resuelven nombres, cómo los puertos identifican servicios y por qué un dispositivo en una red local no es accesible desde internet. Es la base para entender despliegue y conectividad remota.

1.4 Protocolo HTTP: métodos, códigos de estado y JSON

Enseña la estructura de peticiones y respuestas HTTP y el formato JSON para intercambiar datos. Se practica con capturas de tráfico reales.

1.5 Seguridad: exposición de puertos y riesgos de red

Muestra los riesgos de exponer dispositivos y servicios a internet sin protección, junto con buenas prácticas básicas de configuración de red.

Unidad 2. Desarrollo web y APIs REST

Semanas 4–7

Construye la capa de aplicación del proyecto (backend, frontend y persistencia básica) usando IA como herramienta de desarrollo. Cierra con un prototipo desplegado en internet.

2.1 Desarrollo asistido por IA: flujo de trabajo y reglas de uso

Enseña a usar asistentes de IA para generar, revisar y depurar código. Establece las reglas del curso: documentar el uso de IA y poder explicar cada línea entregada.

2.2 Diseño de APIs REST: recursos, endpoints y respuestas

Explica cómo diseñar una API clara: qué recursos expone, qué endpoints tiene y qué respuestas devuelve. Cada equipo diseña la API de su proyecto antes de programarla.

2.3 Desarrollo de backend (FastAPI o Express)

Implementa la API diseñada con un framework moderno. El estudiante debe poder recibir datos del ESP32 y enviarle comandos.

2.4 Persistencia básica con SQLite y SQL fundamental

Introduce las bases de datos relacionales con operaciones básicas de SQL para guardar y consultar lecturas. Es una base sencilla que se amplía en la Unidad 4.

2.5 Frontend: HTML, CSS, JavaScript y consumo de APIs

Enseña a construir una interfaz web que consulta la API y muestra datos del dispositivo. El foco está en entender cómo el frontend se comunica con el backend.

2.6 Diseño responsive y control móvil

Adapta la interfaz para usarse desde el celular y controlar actuadores de forma remota.

2.7 Herramientas de despliegue rápido

Presenta plataformas que permiten publicar un proyecto en internet en minutos, organizadas según el problema que resuelven. El estudiante debe saber elegir la adecuada para su proyecto.

- Backend como servicio: Firebase, Supabase y Appwrite
- Hosting de frontend: Vercel, Netlify y Cloudflare Pages
- Hosting de backend: Railway, Render y Fly.io
- Plataformas IoT listas: Adafruit IO, ThingSpeak y Blynk
- Constructores de apps con IA: Lovable, Bolt y Replit

2.8 Seguridad: autenticación con tokens y variables de entorno

Enseña a proteger la API para que solo usuarios y dispositivos autorizados accedan a ella, y a manejar credenciales fuera del código fuente.

Unidad 3. Conectividad inalámbrica y telemetría

Semanas 8–11

Amplía las opciones de conexión del dispositivo más allá del Wi-Fi e introduce MQTT como protocolo de telemetría. El objetivo es que el estudiante elija la tecnología adecuada según las condiciones reales de su proyecto.

3.1 Criterios de selección de conectividad: alcance, consumo, ancho de banda y costo

Establece los criterios para comparar tecnologías inalámbricas. El estudiante debe justificar técnicamente qué conectividad usa su proyecto.

3.2 Tecnologías de corto alcance: Wi-Fi y Bluetooth Low Energy

Compara Wi-Fi y BLE en consumo, alcance y casos de uso típicos, con práctica de ambas en el ESP32.

3.3 Redes LPWAN: LoRa, LoRaWAN, NB-IoT y Sigfox

Presenta las redes de largo alcance y bajo consumo para aplicaciones en campo, donde no hay Wi-Fi. Destaca sus limitaciones: poco ancho de banda y mensajes pequeños.

3.4 Arquitectura LoRaWAN: nodos, gateways, servidor de red y The Things Network

Explica cómo viaja un dato desde un nodo LoRa hasta internet. Se practica con TTN o, si no hay gateway, con LoRa punto a punto.

3.5 Conectividad celular para IoT: LTE-M y 4G

Cubre el uso de redes móviles para dispositivos que necesitan cobertura amplia y mayor ancho de banda, junto con sus costos operativos.

3.6 Limitaciones del polling y modelo publicador/suscriptor

Muestra por qué consultar constantemente al servidor es ineficiente y cómo el modelo pub/sub resuelve ese problema. Prepara la migración del proyecto a MQTT.

3.7 Broker MQTT: tópicos, wildcards y QoS

Enseña el funcionamiento de un broker, cómo organizar tópicos y qué garantiza cada nivel de QoS. Cada equipo diseña la jerarquía de tópicos de su proyecto.

3.8 Integración de MQTT con backend, actuadores y redes LoRaWAN

Conecta MQTT con el backend de la Unidad 2 para guardar datos y enviar comandos a los actuadores. Muestra cómo TTN entrega datos LoRaWAN por MQTT, lo que unifica ambas arquitecturas.

3.9 Seguridad: autenticación y TLS en el broker

Enseña a proteger el broker con usuario, contraseña y cifrado TLS para evitar que terceros lean o envíen mensajes.

Unidad 4. Datos, integración y despliegue

Semanas 12–15

Da criterio para elegir el tipo de base de datos según el problema, con énfasis en las relacionales como opción por defecto. Cierra con el proyecto desplegado en infraestructura propia.

4.1 Panorama de bases de datos y criterios de selección
Presenta los grandes tipos de bases de datos y los criterios para elegir entre ellos: estructura de los datos, patrones de consulta, consistencia y escalabilidad. El estudiante debe poder justificar la elección para su proyecto.

4.2 Bases de datos relacionales: PostgreSQL
Profundiza en tablas, claves, relaciones, consultas con JOIN y transacciones. Enfatiza por qué suelen ser la opción por defecto cuando los datos tienen estructura y relaciones claras.

4.3 Bases de datos documentales: MongoDB y Firestore
Explica el modelo de documentos flexibles y cuándo conviene frente a un esquema rígido, por ejemplo con datos de estructura variable entre dispositivos.

4.4 Bases de datos clave-valor: Redis
Presenta el almacenamiento clave-valor para accesos muy rápidos, como el último estado de cada dispositivo o una caché. Se muestra como complemento de otra base, no como reemplazo.

4.5 Bases de datos de grafos: Neo4j
Introduce el modelo de nodos y relaciones para datos muy conectados, como redes de dispositivos o dependencias entre componentes.

4.6 Bases de datos de series de tiempo: InfluxDB y Grafana
Explica por qué la telemetría se guarda mejor en bases de series de tiempo, cómo consultarla por ventanas y cómo visualizarla en dashboards de Grafana.

4.7 Docker y despliegue propio en servidor o nube
Enseña a empaquetar el sistema completo en contenedores y desplegarlo en un servidor propio. Se contrasta con las plataformas de la 2.7: más trabajo, pero control total.

6. Ejes transversales

- **Proyecto real.** Los equipos definen su problema al inicio del término y lo mejoran en cada unidad.
- **Desarrollo asistido por IA.** Se usa desde la Unidad 2. Toda entrega exige documentar el uso de IA y poder explicar cada línea del código.
- **Seguridad.** Las unidades 1 a 3 cierran con una subunidad de seguridad aplicada al proyecto.

7. Distribución semanal · II PAO 2026

Clase de 2 horas, lunes de 09:00 a 11:00.

Semana	Fecha	Temas
2	5 oct	**U1.** 1.1 Sistemas telemáticos y arquitectura end-to-end · 1.2 Modelo TCP/IP y direccionamiento
3	12 oct	**U1.** 1.3 Servicios de red: DNS, puertos y NAT · 1.4 Protocolo HTTP · 1.5 Seguridad: exposición de puertos
4	19 oct	**U2.** 2.1 Desarrollo asistido por IA · 2.2 Diseño de APIs REST · 2.3 Desarrollo de backend
5	26 oct	**U2.** 2.4 Persistencia con SQLite · 2.5 Frontend y consumo de APIs · 2.6 Diseño responsive y control móvil
6	2 nov	Feriado
7	9 nov	**U2.** 2.7 Herramientas de despliegue rápido · 2.8 Seguridad: autenticación y variables de entorno
8	16 nov	**U3.** 3.1 Criterios de selección de conectividad · 3.2 Wi-Fi y Bluetooth Low Energy
9	23 nov	**U3.** 3.3 Redes LPWAN · 3.4 Arquitectura LoRaWAN · 3.5 Conectividad celular para IoT
10	30 nov	**U3.** 3.6 Polling y modelo publicador/suscriptor · 3.7 Broker MQTT: tópicos, wildcards y QoS
11	7 dic	**U3.** 3.8 Integración de MQTT con backend, actuadores y LoRaWAN · 3.9 Seguridad en el broker
12	14 dic	**U4.** 4.1 Panorama de bases de datos · 4.2 Relacionales: PostgreSQL · 4.3 Documentales: MongoDB y Firestore
13	21 dic	**U4.** 4.4 Clave-valor: Redis · 4.5 Grafos: Neo4j · 4.6 Series de tiempo: InfluxDB y Grafana
14	28 dic	Vacaciones
15	4 ene	**U4.** 4.7 Docker y despliegue propio en servidor o nube
16	13 ene	Segunda evaluación: demo final del proyecto
18	27 ene	Tercera evaluación (mejoramiento)

8. Prácticas de laboratorio

10 prácticas de 2 horas, guiadas por la profesora de laboratorio. Cada práctica tiene como objetivo usar una herramienta real y desarrollar una habilidad ligada al contenido teórico.

Lab 1. Diagnóstico de red

Semana 3 · 12 al 16 de octubre

Objetivo: Identificar la configuración de red de un equipo y ver los dispositivos conectados.

Herramientas: ipconfig/ifconfig, ping, nslookup, panel de administración del router y Wireshark.

Hardware: PC, router del laboratorio y ESP32.

Al terminar, el estudiante entrega:

- La IP, máscara, gateway y DNS de su PC, explicando qué hace cada uno.
- El ESP32 conectado al router, identificado en la lista de clientes DHCP del panel y verificado con ping.
- Una captura de Wireshark con un ping y una consulta DNS, señalando origen y destino.

Lab 2. Peticiones HTTP con Postman

Semana 4 · 19 al 23 de octubre

Objetivo: Construir, probar y depurar peticiones HTTP antes de programarlas en un dispositivo.

Herramientas: Postman (o Bruno), webhook.site y Wireshark.

Hardware: ESP32 y DHT11.

Al terminar, el estudiante entrega:

- Una colección de Postman con peticiones GET y POST a una API pública, con sus códigos de respuesta interpretados.
- El ESP32 enviando lecturas del DHT11 en JSON por POST, visibles en webhook.site.
- Dos capturas de Wireshark que comparen la misma petición por HTTP y por HTTPS.

Lab 3. Estructura de un backend

Semana 5 · 26 al 30 de octubre

Objetivo: Entender cómo funciona un backend leyendo, ejecutando y modificando código provisto por el curso. En esta práctica no se usa IA.

Herramientas: VS Code, FastAPI con su documentación Swagger, Git y GitHub.

Hardware: ESP32, DHT11 y LED.

Al terminar, el estudiante entrega:

- La API provista corriendo localmente, con todos sus endpoints probados desde Swagger.
- El ESP32 enviando lecturas y encendiendo el LED según el comando que devuelve la API.
- El código subido a su propio repositorio de GitHub.

Reto: Agregar a mano un endpoint nuevo, por ejemplo uno que devuelva el promedio de las últimas lecturas.

Lab 4. Persistencia y dashboard

Semana 7 · 9 al 13 de noviembre

Objetivo: Guardar datos del dispositivo y mostrarlos en una interfaz usable desde el celular.

Herramientas: SQLite con DB Browser, Chart.js y el modo responsive de DevTools.

Hardware: ESP32, DHT11 y relé o LED.

Al terminar, el estudiante entrega:

- Las lecturas guardadas en SQLite y consultables desde DB Browser.
- Un dashboard con gráfico histórico y un botón que acciona el actuador.
- El dashboard funcionando desde el celular conectado a la misma red.

Lab 5. Despliegue en la nube con autenticación

Semana 8 · 16 al 20 de noviembre

Objetivo: Publicar el sistema en internet y protegerlo con token.

Herramientas: Vercel, Render o Supabase, archivos .env y GitHub.

Hardware: ESP32 y DHT11.

Al terminar, el estudiante entrega:

- El frontend accesible desde una URL pública.
- El ESP32 enviando datos a la nube con token.
- La prueba de que una petición sin token es rechazada (código 401).
- Un repositorio sin credenciales en el código.

Lab 6. LoRa en campo

Semana 10 · 30 de noviembre al 4 de diciembre

Objetivo: Establecer un enlace LoRa y entender la relación entre alcance, velocidad y tamaño de mensaje.

Herramientas: librería RadioLib y monitor serial con registro de datos.

Hardware: Pares de ESP32 con LoRa y DHT11.

Al terminar, el estudiante entrega:

- Un enlace punto a punto con un nodo transmisor y uno receptor.
- Una tabla de RSSI y SNR en varios puntos del campus, comparando spreading factor 7 y 12.
- La lectura enviada como bytes compactos en lugar de JSON, con una explicación de por qué es necesario en LoRa.

Lab 7. Fundamentos de MQTT

Semana 11 · 7 al 11 de diciembre

Objetivo: Operar MQTT con distintos clientes y diseñar una jerarquía de tópicos.

Herramientas: Mosquitto en la PC, mosquitto_pub/mosquitto_sub, MQTT Explorer y la librería PubSubClient.

Hardware: PC, ESP32 y DHT11.

Al terminar, el estudiante entrega:

- Mensajes publicados y recibidos desde la terminal.
- El ESP32 publicando lecturas en un broker común del laboratorio, con tópicos por grupo (por ejemplo, lab/grupo3/temperatura).
- Una suscripción con wildcards que reciba los datos de todos los grupos a la vez, vista en MQTT Explorer.
- Una prueba comparando QoS 0 y QoS 1 al desconectar y reconectar un cliente.

Lab 8. Control y seguridad con MQTT

Semana 12 · 14 al 18 de diciembre

Objetivo: Usar MQTT en ambos sentidos (telemetría y comandos), integrarlo con un backend y asegurar el broker.

Herramientas: Mosquitto en Raspberry Pi, MQTT Explorer y la librería paho-mqtt de Python.

Hardware: Raspberry Pi, dos ESP32, DHT11 y relé o LED.

Al terminar, el estudiante entrega:

- Un ESP32 publicando lecturas y otro suscrito a un tópico de comandos que acciona el actuador.
- Un script en Python suscrito a las lecturas que las guarda en SQLite.
- El broker configurado con usuario y contraseña, y la prueba de que rechaza clientes sin credenciales.

Lab 9. Integración con Node-RED y PostgreSQL

Semana 13 · 21 al 25 de diciembre

Objetivo: Integrar varias tecnologías en un flujo de datos completo sin programar todo desde cero.

Herramientas: Node-RED, PostgreSQL y pgAdmin.

Hardware: Nodo ESP32 LoRa con sensor, ESP32 LoRa receptor y Raspberry Pi.

Al terminar, el estudiante entrega:

- El flujo completo funcionando: nodo LoRa, receptor que publica por MQTT, Node-RED y PostgreSQL.
- Las tablas de dispositivos y lecturas modeladas con sus claves.
- Una consulta con JOIN que muestre las últimas lecturas por dispositivo.

Lab 10. Stack de telemetría con Docker

Semana 15 · 4 al 8 de enero

Objetivo: Levantar y desplegar un stack completo con un solo archivo.

Herramientas: Docker, docker-compose, InfluxDB y Grafana.

Hardware: Raspberry Pi (o un VPS) y ESP32 con sensor.

Al terminar, el estudiante entrega:

- Un docker-compose.yml que levanta Mosquitto, InfluxDB y Grafana con un solo comando.
- La telemetría del ESP32 guardada en InfluxDB.
- Un dashboard de Grafana con al menos un gráfico en vivo y una consulta por ventana de tiempo.

Semanas sin laboratorio: semana 2 (aún no hay teoría), semana 6 (feriado), semana 9 y semana 14 (vacaciones).

9. Métodos de evaluación

- Proyectos
- Laboratorio/Experimental
- Exámenes
- Lecciones
- Tareas
- Participación
- Otras: defensa oral del código del proyecto

10. Bibliografía

Lectura obligatoria

- Kurose, J., Ross, K. *Computer Networking: A Top-Down Approach* (8.ª edición).
- Hanes, D., Salgueiro, G., Grossetete, P., Barton, R., Henry, J. *IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things* (1.ª edición).

Lectura adicional, recursos de software e Internet

- Guinard, D., Trifa, V. *Building the Web of Things* (1.ª edición).
- Lea, P. *IoT and Edge Computing for Architects* (2.ª edición).
- Kleppmann, M. *Designing Data-Intensive Applications* (1.ª edición).
- MDN Web Docs: documentación de HTML, CSS y JavaScript.
- Documentación oficial de FastAPI, Eclipse Mosquitto, The Things Network, InfluxDB y Docker.
- HiveMQ, serie *MQTT Essentials* (recurso en línea).